

SIMATS ENGINEERING
CO 2- ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1103

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.

(BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 6 Total Marks:30

Annexure A

Scenario-Based

Code of Conduct:

I GIRIJA B / 192311044 certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

Scenario-Based

ANNEXURE A – SCENARIO-BASED ANSWERS

1. Use Case Diagram – Smart Pharmacy Management System

Actors

The main actors are:

1. **Customer**
2. **Pharmacist**
3. **Administrator**
4. **Payment Gateway** – external system

Main Use Cases

Customer

- Register/Login
- Search Medicines
- View Medicine Details
- Upload Prescription
- Place Medicine Order
- Make Online Payment
- Track Order
- View Order Status

Pharmacist

- Login
- Verify Prescription
- Approve Prescription
- Reject Prescription
- Dispense Medicines
- Update Order Status

Administrator

- Manage Medicine Inventory

- Add Medicine
- Update Medicine
- Remove Medicine
- Manage User Accounts

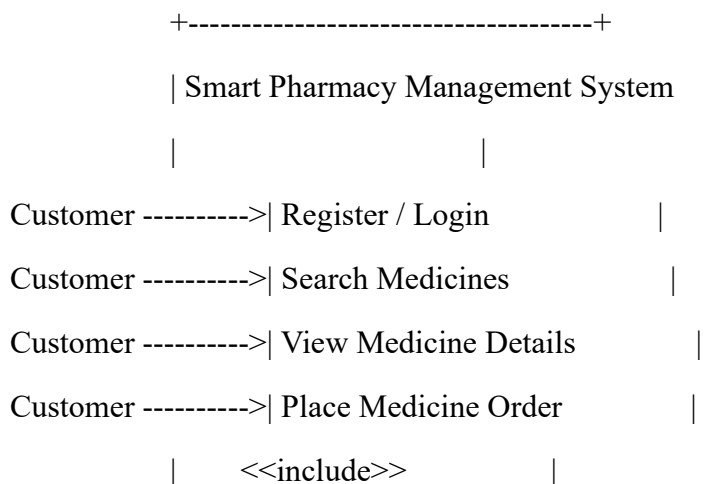
Payment Gateway

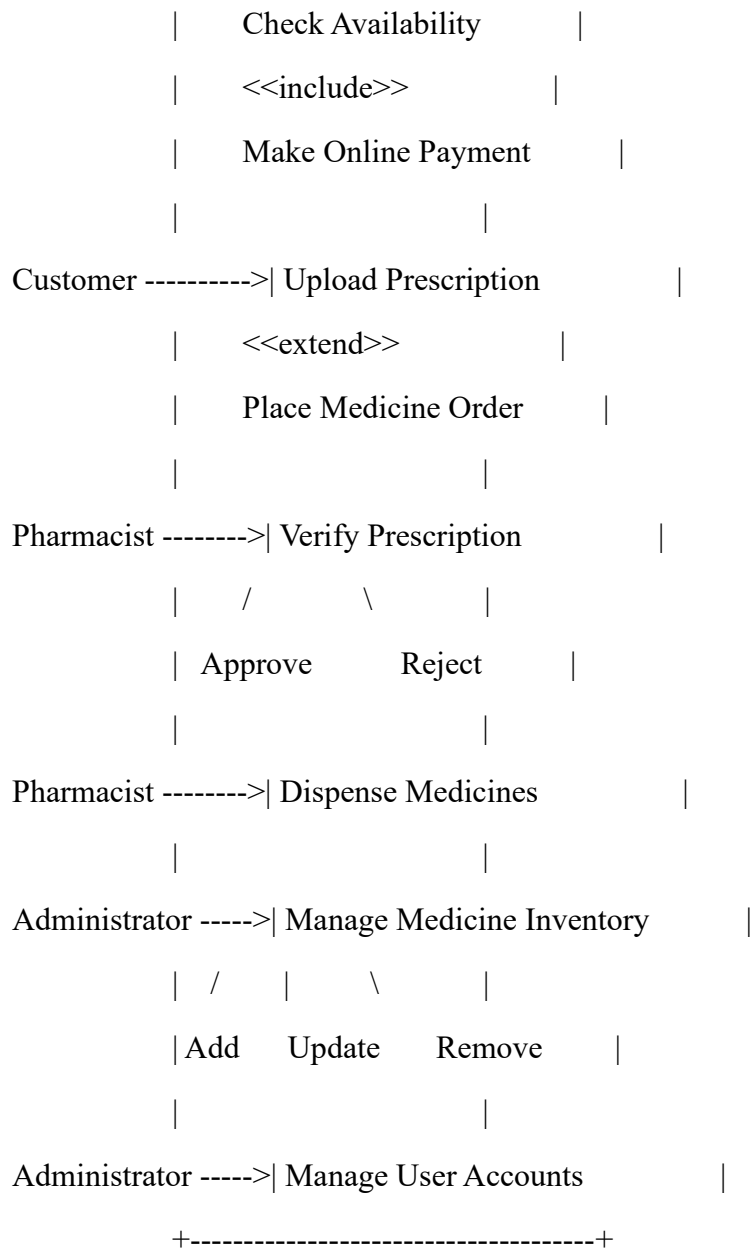
- Process Payment
- Confirm Payment

Relationships

- **Place Medicine Order** <<include>> **Check Medicine Availability**
- **Place Medicine Order** <<include>> **Make Online Payment**
- **Make Online Payment** <<include>> **Process Payment**
- **Upload Prescription** <<extend>> **Place Medicine Order** when prescription medicine is required.
- **Verify Prescription** is generalized into:
 - Approve Prescription
 - Reject Prescription
- **Manage Medicine Inventory** is generalized into:
 - Add Medicine
 - Update Medicine
 - Remove Medicine.

Use Case Diagram Representation





Payment Gateway ---> Process Payment

Explanation

The **Customer** searches for medicines, uploads prescriptions when required, places orders, and makes payments. The **Pharmacist** verifies prescriptions and dispenses medicines. The **Administrator** manages inventory and user accounts. The payment gateway is an external actor that processes online payments.

The <<include>> relationship is used for compulsory functionality. The <<extend>> relationship is used for optional or conditional behavior. Generalization is used when a general use case has specialized use cases.

2. Class Diagram – Smart Hotel Management System

Required Classes

1. Guest

Attributes:

- guestId
- name
- email
- phone
- address

Methods:

- + register()
 - + login()
 - + searchRoom()
 - + makeReservation()
 - + cancelReservation()
-

2. Room

Attributes:

- roomNumber
- roomType
- price
- status

Methods:

- + checkAvailability()
 - + updateStatus()
 - + getRoomDetails()
-

3. Reservation

Attributes:

- reservationId
- checkInDate
- checkOutDate
- status

Methods:

- + createReservation()
 - + modifyReservation()
 - + cancelReservation()
 - + confirmReservation()
-

4. Payment

Attributes:

- paymentId
- amount
- paymentDate
- paymentMethod
- paymentStatus

Methods:

- + makePayment()
 - + verifyPayment()
 - + generateReceipt()
-

5. Receptionist

Attributes:

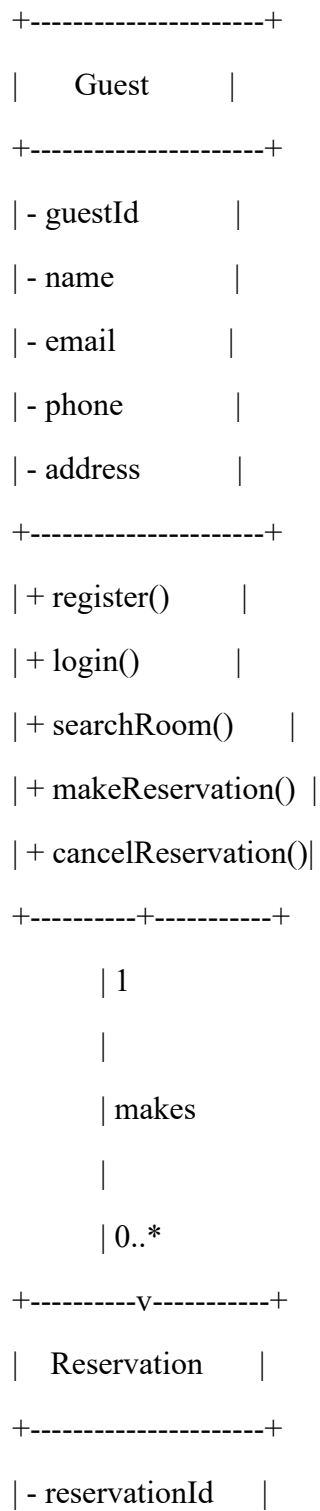
- employeeId
- name
- phone

Methods:

- + checkInGuest()

+ checkOutGuest()
+ manageReservation()
+ assignRoom()

Class Diagram



```

| - checkInDate      |
| - checkOutDate     |
| - status           |
+-----+
| + createReservation()|
| + modifyReservation()|
| + cancelReservation()|
| + confirmReservation()|
+-----+-----+
      | 1
      |
      | includes
      |
      | 1..*
+-----v-----+
|   Room   |
+-----+
| - roomNumber      |
| - roomType        |
| - price           |
| - status          |
+-----+
| + checkAvailability()|
| + updateStatus()   |
| + getRoomDetails() |
+-----+

```

Reservation 1 ----- 1..* Payment

```

|

```


| composition

|

+---- Payment

Receptionist 1 ----- 0..* Reservation

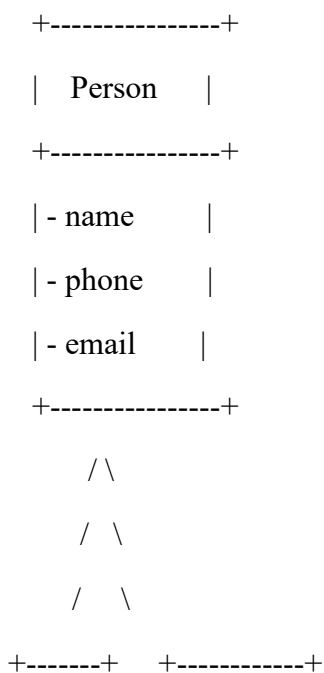
manages

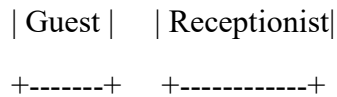
Relationships

- **Guest — Reservation:** Association
One guest can make zero or many reservations.
- **Reservation — Room:** Association
A reservation may include one or more rooms.
- **Reservation — Payment:** Composition
A payment is strongly associated with a reservation.
- **Hotel — Room:** Aggregation
A hotel contains many rooms, but rooms can conceptually exist independently of a particular hotel system.
- **Receptionist — Reservation:** Association
A receptionist manages reservations.

Inheritance

A general superclass can be introduced:





Thus, Guest and Receptionist inherit common properties from Person.

Multiplicity

- One Guest → 0..* Reservations
 - One Reservation → 1..* Rooms
 - One Reservation → 1 or more Payments
 - One Receptionist → 0..* Reservations
-

3. Sequence Diagram – Online Banking System

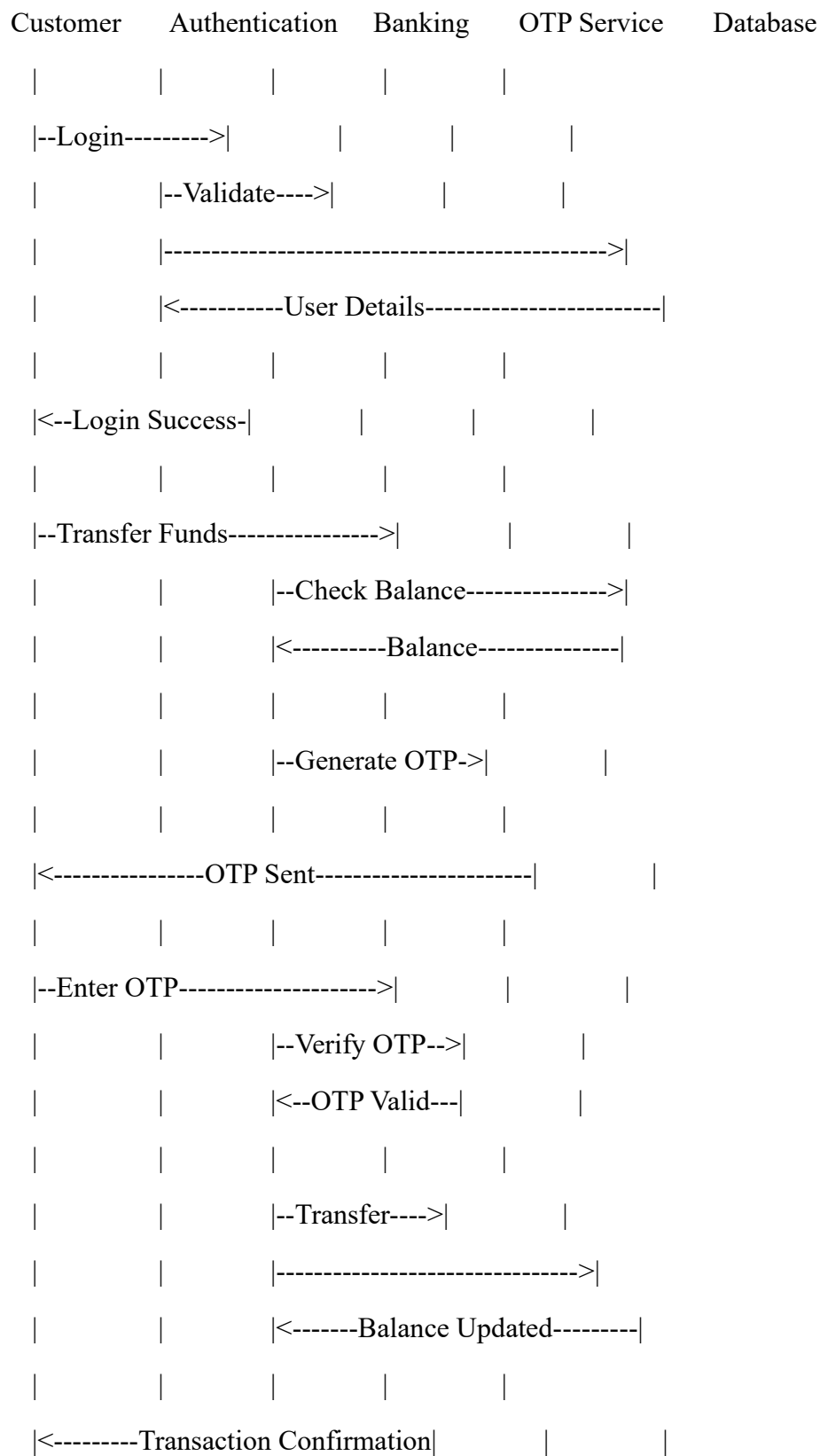
Participating Objects

1. **Customer**
2. **Authentication Service**
3. **Banking Server**
4. **OTP Service**
5. **Database**

Sequence of Operations

1. Customer enters login details.
2. Authentication Service validates the credentials.
3. Authentication Service requests user details from the Database.
4. Login is successful.
5. Customer requests a fund transfer.
6. Banking Server validates the account and balance.
7. OTP is generated and sent to the customer.
8. Customer enters the OTP.
9. OTP is verified.
10. Banking Server performs the transfer.
11. Database updates account balances.
12. Confirmation is sent to the customer.

Sequence Diagram



Explanation

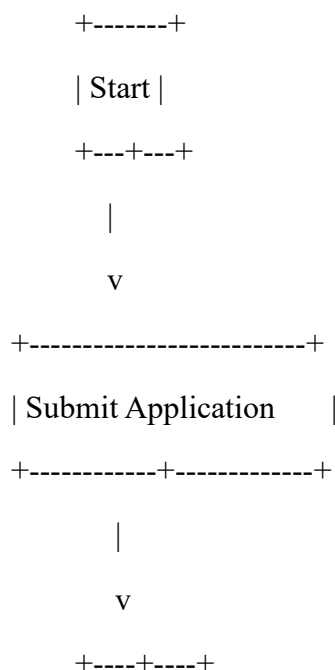
The sequence diagram shows the chronological order of communication. The customer first authenticates, then requests a fund transfer. The banking server verifies the account and balance. An OTP is generated and verified. Only after successful OTP verification does the banking server update the database and confirm the transaction.

4. Activity Diagram – Passport Application System

Main Activities

1. Start
2. Submit Application
3. Upload Supporting Documents
4. Identity Verification
5. Pay Processing Fee
6. Application Review
7. Approve or Reject Application
8. Passport Processing
9. Passport Dispatch
10. Receive Passport
11. End

Activity Diagram



| Fork |

+-----+-----+

/\

/ \

v v

+-----+ +-----+

| Upload Documents| | Identity Verification|

+-----+-----+ +-----+-----+

\ /

\ /

v v

+-----+

| Join |

+-----+-----+

|

v

+-----+

| Pay Processing Fee |

+-----+-----+

|

v

+-----+

| Review Application |

+-----+-----+

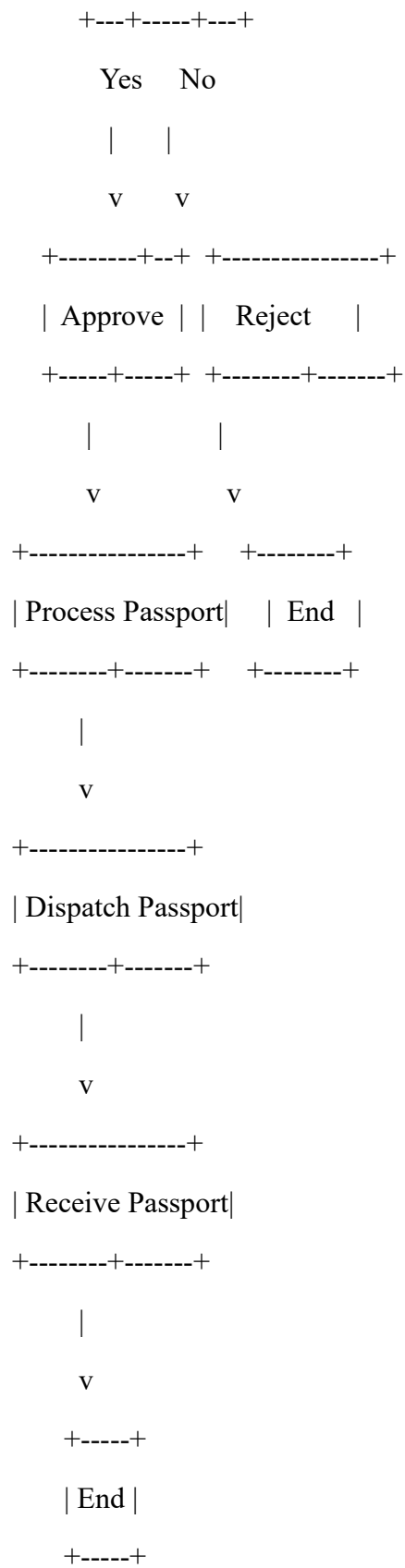
|

v

+-----+

| Documents |

| Valid? |



Explanation

The **fork** allows supporting document upload and identity verification to occur concurrently. The **join** ensures that both activities are completed before the application proceeds.

The decision node checks whether the application and documents are valid:

- If valid → Application is approved.
- If invalid → Application is rejected.

This activity diagram clearly represents workflow, decision-making, and parallel processing.

5. State Diagram – Smart Elevator Control System

States

1. **Idle**
2. **Door Open**
3. **Door Closed**
4. **Moving Up**
5. **Moving Down**
6. **Emergency Stop**
7. **Maintenance Mode**

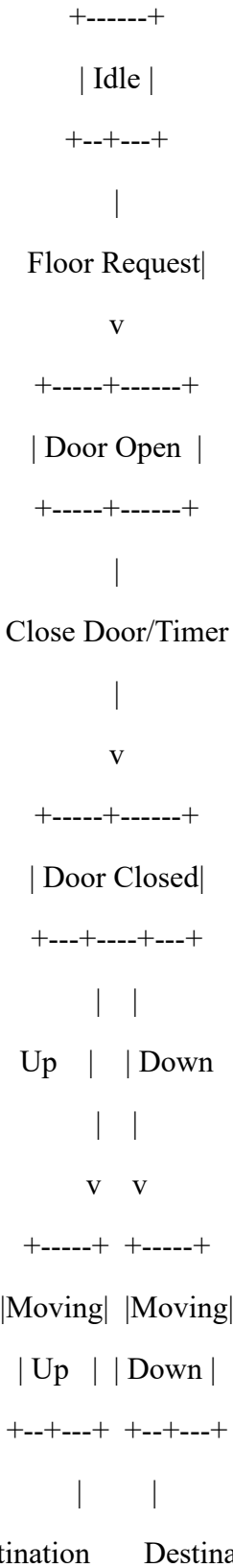
State Transition Table

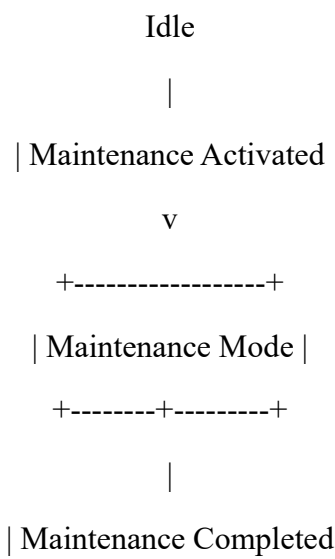
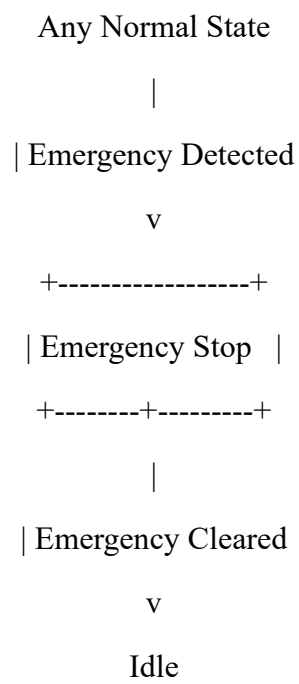
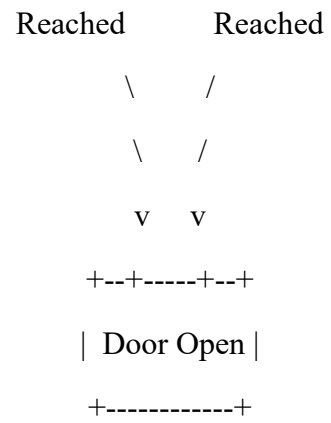
Current State	Event	Next State
Idle	Floor request	Door Open
Door Open	Close button/timer	Door Closed
Door Closed	Up request	Moving Up
Door Closed	Down request	Moving Down
Moving Up	Destination reached	Door Open
Moving Down	Destination reached	Door Open
Any Normal State	Emergency detected	Emergency Stop
Emergency Stop	Emergency cleared	Idle
Idle	Maintenance activated	Maintenance Mode

Current State	Event	Next State
---------------	-------	------------

Maintenance Mode	Maintenance completed	Idle
------------------	-----------------------	------

State Transition Diagram





Explanation

The elevator normally remains in the **Idle** state. When a floor is requested, the elevator opens or closes the doors and then moves in the required direction.

When the destination floor is reached, the elevator stops and opens the doors. If an emergency occurs at any time, the system enters the **Emergency Stop** state. The elevator can return to normal operation only after the emergency is cleared.

During maintenance, the elevator enters **Maintenance Mode** and does not accept normal passenger operations.

6. Deployment Diagram – Smart Home Automation System**Hardware Nodes****1. Smart Sensors**

- Temperature Sensor
- Motion Sensor
- Smoke Sensor
- Door Sensor

2. IoT Controller

- Controls smart devices.

3. Home Gateway/Router

- Provides local network communication.

4. Cloud Server

- Stores data and provides remote services.

5. Mobile Device

- Android or iOS application.

6. Smart Home Devices

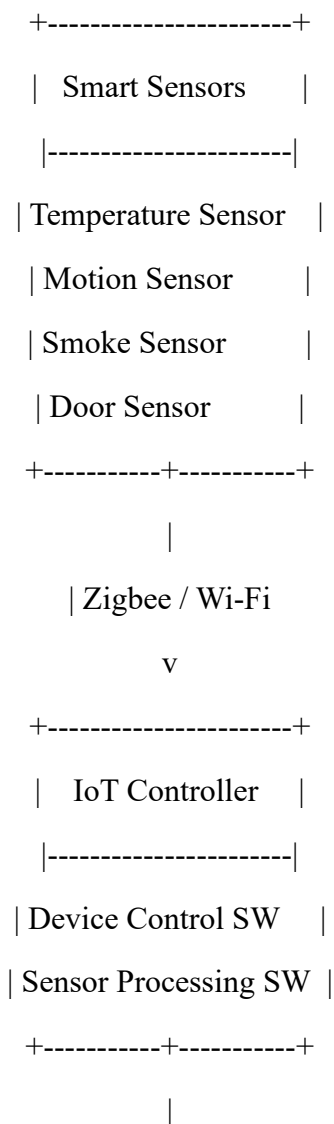
- Smart Lights
- Smart Locks
- Smart Appliances

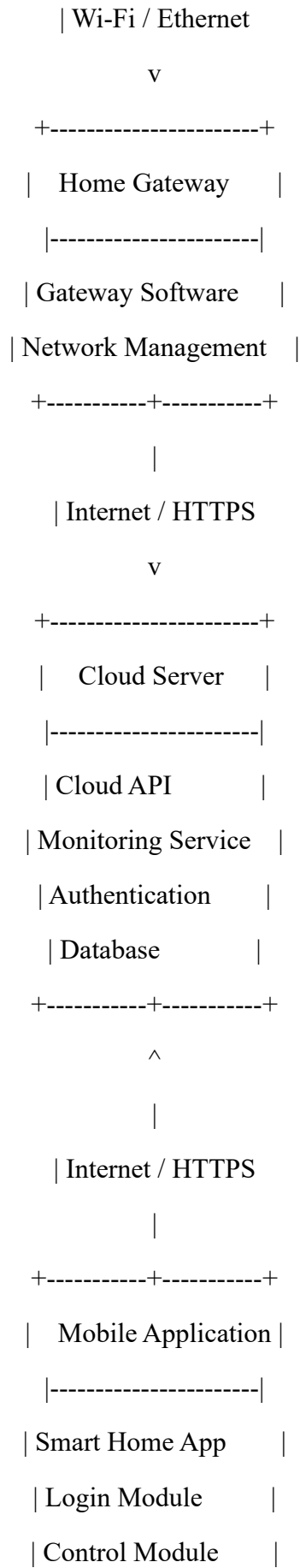
- Security Alarm

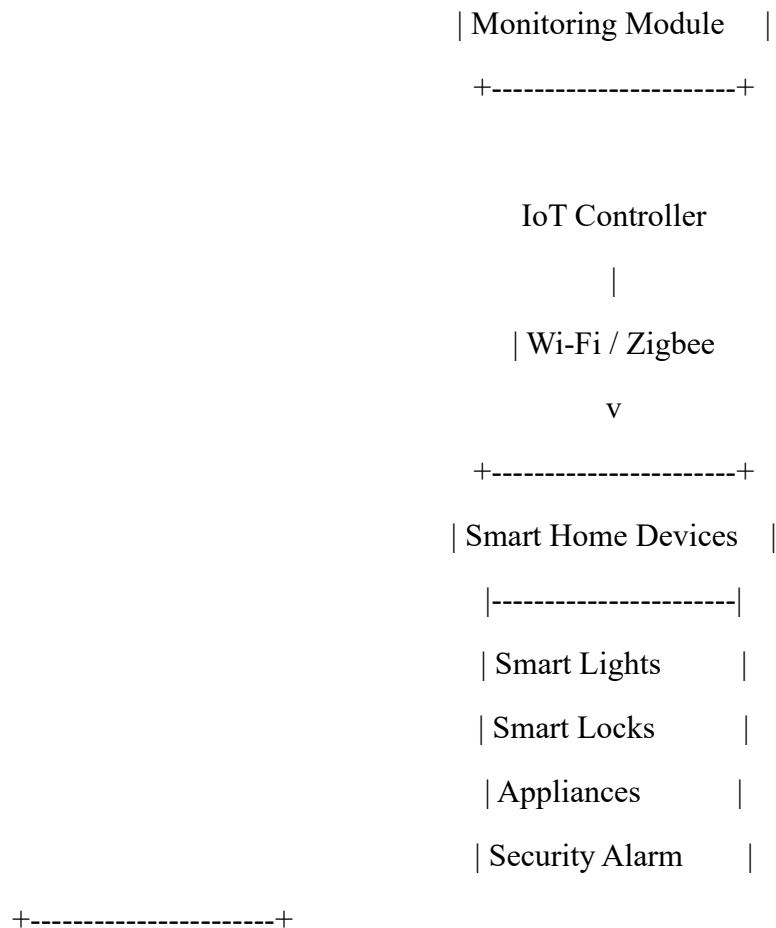
Software Artifacts

- Smart Sensor Firmware
 - IoT Control Software
 - Home Gateway Software
 - Cloud Monitoring Service
 - Database
 - Mobile Application
-

Deployment Diagram







Explanation

The **Smart Sensors** collect data from the home environment. This information is sent to the **IoT Controller**, which processes sensor data and controls smart devices.

The **Home Gateway** connects local IoT devices to the Internet. The **Cloud Server** provides authentication, data storage, monitoring, and remote control services.

The **Mobile Application** communicates with the cloud server through the Internet. Users can remotely monitor sensors and control lights, locks, and appliances.

The deployment diagram clearly represents:

- Physical hardware nodes.
- Software deployed on each node.
- Communication channels.
- Network connections.
- The complete physical architecture of the smart home system.